EXPERIMENT - 01 : Cryptography in Blockchain, Merkle root Tree Hash

AIM : To study the concepts of Cryptography in Blockchain, Cryptographic Hash Functions, Proof-of-Work, and Merkle Trees, and to implement SHA-256 hashing, nonce-based cryptographic puzzles, and Merkle Root Hash generation using Python.

THEORY :

Cryptography in Blockchain

Cryptography is one of the fundamental technologies used to secure blockchain networks. It protects transaction data, verifies the authenticity of information, and ensures that stored data cannot be easily modified.

Blockchain mainly uses cryptographic hash functions and digital signatures. A hash function converts input data of any size into a fixed-length string called a hash value. Even a small change in the input produces a completely different hash.

In this experiment, the SHA-256 hashing algorithm from Python's **hashlib** library is used.

Cryptographic Hash Function

A cryptographic hash function has the following properties:

- It accepts data of any size as input.
- It produces a fixed-length hash.
- The same input always produces the same hash.
- It is computationally difficult to find the original input from its hash.
- A small change in the input produces a significantly different hash.
- It is computationally difficult to find two different inputs with the same hash.

SHA-256 produces a 256-bit hash, normally represented as a 64-character hexadecimal string.

Cryptographic Puzzle and Golden Nonce

In blockchain systems that use Proof-of-Work, miners have to solve a cryptographic puzzle. They repeatedly change a value called a nonce and calculate the hash until the resulting hash satisfies a predefined condition.

For example, if the difficulty requires one leading zero, the hash must look like:

0abc1234...

If two leading zeros are required:

00f7a91c...

The Golden Nonce is the nonce value that produces a hash satisfying the required condition.

Merkle Tree

A **Merkle Tree** is a tree-like data structure used in blockchain to efficiently represent and verify a large number of transactions.

Each transaction is first converted into a cryptographic hash. Pairs of hashes are then combined and hashed again. This process continues until only one hash remains. This final hash is called the **Merkle Root**.

INPUT:

Program #1: Python program that uses the hashlib library to create the hash of a given string:

XXXXXXX #error statement; 3 Error

```
import hashlib                                                  #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                           #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                          #error statement
```

OUTPUT:

```
Enter a string: user
Hash: 04f8996da763b7a969b1028ee3007569eaf3a635486ddab211d512c85b9df8fb
```

INPUT:

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                                    # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                          # Error
```

OUTPUT:

```
Enter a string: user
Enter the nonce: 10
Hash Code: 5bbf1a9e0de062225a1bb7df8d8b3719591527b74950810f16b1a6bc6d7bd29b
```

INPUT:

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string 3 Error

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)                      #Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):                          #Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1
```

```
# Get user input
input_string = "A"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)                                    #Error
```

OUTPUT:

```
Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb
Input String: A
Leading Zeros: 1
Expected Nonce: 9
```

INPUT:

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib

def hash_transaction(transaction):
    return hashlib.sha256(transaction.encode('utf-8')).hexdigest()


def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Hash all transactions first
    transactions = [hash_transaction(transaction) for transaction in transactions]

    print("Initial Transaction Hashes:")
    for i, transaction_hash in enumerate(transactions):
        print(f"T{i + 1}: {transaction_hash}")

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
```

```
        # Recursive construction of the Merkle Tree
        while len(transactions) > 1:

            # If number of hashes is odd, duplicate the last hash
            if len(transactions) % 2 != 0:
                transactions.append(transactions[-1])

            new_transactions = []

            for i in range(0, len(transactions), 2):

                # Combine two hashes
                combined = transactions[i] + transactions[i + 1]

                # Hash the combined value
                hash_combined = hashlib.sha256(
                    combined.encode('utf-8')
                ).hexdigest()

                # Store the new hash
                new_transactions.append(hash_combined)
```

```
                # Print intermediate hash
                print(f"\nCombining:")
                print(f"Hash 1: {transactions[i]}")
                print(f"Hash 2: {transactions[i + 1]}")
                print(f"New Hash: {hash_combined}")

            transactions = new_transactions

        return transactions[0]


    # Sample Transactions
    transactions = [
        "Alice -> Bob : $200",
        "Bob -> Dave : $500",
        "Dave -> Eve : $100",
        "Eve -> Alice : $300",
        "Roo -> Bob : $50"
    ]

    # Build Merkle Tree
    merkle_root = build_merkle_tree(transactions)
```

OUTPUT:

```
Initial Transaction Hashes:
T1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
T2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
T3: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
T4: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
T5: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Combining:
Hash 1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
Hash 2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
New Hash: 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

Combining:
Hash 1: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
Hash 2: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
New Hash: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Combining:
Hash 1: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
Hash 2: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
New Hash: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
```

```
Combining:
Hash 1: 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
Hash 2: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
New Hash: ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Combining:
Hash 1: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Hash 2: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
New Hash: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Combining:
Hash 1: ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
Hash 2: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45
New Hash: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Merkle Root: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
```

CONCLUSION :

The experiment successfully demonstrated SHA-256 hashing, Proof-of-Work, Golden Nonce, and Merkle Tree construction using Python. It showed how cryptographic hashing ensures data security and integrity, while Merkle Trees enable efficient transaction verification in blockchain.